

Advanced Predictive Analytics 2025/2026 — Course Project

Weight: 40% of final grade | **Teams:** 4 students **Start:** Friday, April 10, 2026 | **End:** Wednesday, May 20, 2026

1. Overview

In this project you will build an **AI-powered autonomous research agent** that designs, trains, evaluates, and iterates on deep learning models — without human intervention. The agent uses a **locally-hosted Large Language Model** as its “brain” to drive the machine learning experimentation loop.

You will choose one of two Kaggle competitions as the target task for your agent.

In short: you are building a machine that builds machines.

A note on development. This project is ambitious by design. You are **strongly encouraged** to use LLM-powered development tools to help you build the agent itself. Any modern coding assistant will work — we recommend experimenting with tools such as **ChatGPT Codex**, **Claude Code**, **Gemini CLI**, or **Cursor**. These tools can help you scaffold the agent loop, debug issues, and iterate faster. The project would be very difficult without them. So you are allowed to use them. However, in the end, you need to understand everything that was done and maybe called to justify the coding decisions. Avoid to overengineer a solution that you will not understand.

2. The Task

2.1 Choose Your Competition Track

Each team must choose **one** of the two Kaggle competitions below as the target ML problem for their agent. Read both descriptions carefully before deciding.

Track A — NLP with Disaster Tweets (Standard) **Competition:** <https://www.kaggle.com/competitions/nlp-getting-started>

Twitter (now X) has become an important communication channel during emergencies. But not every tweet that mentions “fire” or “crash” is about a real disaster. Your task is to build a binary classifier that predicts whether a given tweet is about a **real disaster** (target = 1) or **not** (target = 0).

- **Dataset:** ~10,000 labeled tweets (train), ~3,200 unlabeled tweets (test)
- **Evaluation metric:** F1 score
- **Submission format:** CSV with columns `id` and `target`
- **Competition status:** Permanently open — unlimited submissions

Course content exercised:

Approach	Course content
Bag-of-words / n-grams + Dense	MLPs
1D Convolutions on text sequences	CNNs
LSTM / GRU / Bidirectional RNNs	RNNs
Learned embeddings vs. pretrained (GloVe)	Deep learning for text
Transformer / self-attention	Language models

Why choose this track: The dataset is small and trains fast even on CPU. The agent can run many experiments quickly. The task covers the full range of NLP techniques from the course. This is the safer, more accessible option.

Track B — BirdCLEF+ 2026: Bird Species Recognition from Audio (Advanced) Competition: <https://www.kaggle.com/competitions/birdclef-2026> **Prize pool:** \$50,000 | **Entry deadline:** May 27, 2026

Identify wildlife species from their vocalizations in audio recordings collected across the Pantanal wetlands in South America. The Pantanal hosts over 650 bird species and faces threats from wild-fires, agricultural expansion, and climate change. Your models will support biodiversity monitoring in this ecosystem.

- **Dataset:** Audio recordings from a network of 1,000 acoustic recorders across the Pantanal
- **Task:** For each 5-second audio window, predict the probability of presence for each of 234 species
- **Evaluation metric:** Macro-averaged ROC-AUC (skipping classes with no true positives)
- **Submission format:** CPU-only Kaggle notebook with a 90-minute runtime limit
- **Competition status:** Active featured competition

Course content exercised:

Approach	Course content
Audio -> mel-spectrogram -> image	Signal preprocessing
2D CNNs on spectrograms (Conv2D, pooling)	CNNs
Transfer learning from pretrained models	Transfer learning, data augmentation
Multi-label classification with 234 species	Model design

Why choose this track: This is a real, active competition with a \$50,000 prize pool hosted by the Cornell Lab of Ornithology. The data is messy, real-world field recordings — far more challenging than curated datasets. Choose this track if your team has access to adequate hardware (16+ GB RAM, ideally a GPU or GCP credits) and is willing to take on a harder problem.

BirdCLEF is harder and will be valued accordingly. This track involves real-world noisy data, a 234-class multi-label problem, heavier compute requirements, and a CPU-only submission con-

straint. Teams that choose BirdCLEF and produce a working solution will be graded more favorably than Disaster Tweets teams with a comparable level of effort.

2.2 The Autonomous Research Agent

Regardless of which track you choose, the core of the project is the **agent loop**. Your agent must:

1. **Load and explore** the competition training data
2. **Propose** a model architecture and training configuration (via the local LLM)
3. **Generate** executable Python code (TensorFlow/Keras or PyTorch)
4. **Execute** the generated code in a sandboxed environment
5. **Capture** training results (loss, metrics, learning curves)
6. **Feed results back** to the LLM for analysis — what worked? what didn't? why?
7. **Iterate** — the LLM proposes a new experiment based on what it learned
8. **Produce** a Kaggle submission file when a satisfactory model is found

The agent should run **autonomously** — once started, it should require no human input to complete a full experimentation cycle. You decide how many iterations it runs, how it manages its “memory” of past experiments, and what stopping criteria it uses.

Design considerations: - How do you structure the prompt(s) so the LLM proposes sensible architectures? - How do you pass training results back to the LLM (raw numbers? plots? summaries?)? - How do you prevent the LLM from repeating failed experiments? - How do you handle code that fails to execute (syntax errors, Memory Errors, etc.)? - Should the agent maintain a log of all experiments and their outcomes? - Should you have only one agent or multiple ones? - Should the agent be allowed to write all parts of the code or only constraint parts of it?

Track-specific agent considerations:

- **Track A (Disaster Tweets):** The agent should explore text preprocessing strategies (tokenization, n-grams, sequence encoding), embedding strategies (learned vs. pretrained), and a range of architectures from dense networks to Transformers.
 - **Track B (BirdCLEF):** The agent should explore audio preprocessing (mel-spectrogram parameters, window sizes), CNN architectures, transfer learning from pretrained models, data augmentation strategies, and multi-label classification heads. The agent must also produce a submission notebook that meets the 90-minute CPU runtime constraint.
-

3. Getting Started

3.1 Local LLM Infrastructure

Your agent needs a locally-running LLM. **Any modern open-weight model will work.** The only requirement is that the model runs locally (or on your own cloud VM) and the agent works end-to-end. Below are the main tools to run local models, from easiest to most hands-on. **We recommend starting with Ollama.**

For your reference, I am running a baseline version of the project in an old macbook pro (2017) with an intel CPU and 16Gb of Ram. The agent is not fast, but completely fulfills the project objectives.

Ollama (Recommended) The simplest way to run LLMs locally. One command to install, one command to run.

- **Install:** <https://ollama.com> — download the installer for macOS, Windows, or Linux
- **Run a model:**

```
ollama run gemma4          # Google Gemma 4 E4B (~9.6 GB)
ollama run qwen3.5:9b      # Alibaba Qwen 3.5 9B (~6 GB)
ollama run qwen3-coder     # Alibaba Qwen 3 Coder – optimized for code
ollama run glm-4.7-flash   # Zhipu GLM-4.7 – 198K context
ollama run deepseek-r1:8b  # DeepSeek R1 – reasoning-focused
ollama run phi4:14b        # Microsoft Phi-4 – strong reasoning
```

Browse all available models at <https://ollama.com/library>

- **Python integration:**

```
pip install ollama
```

```
import ollama
response = ollama.chat(model='gemma4', messages=[
    {'role': 'user', 'content': 'Write a Keras model for binary text classification'}
])
print(response['message']['content'])
```

- Ollama also exposes an **OpenAI-compatible API** at <http://localhost:11434>, so you can use the `openai` Python library as a client. This makes it easy to swap models without changing your agent code.

LM Studio A desktop GUI application for running LLMs. Good if you prefer a visual interface.

- **Install:** <https://lmstudio.ai>
- Browse and download hundreds of models from within the app
- Exposes an OpenAI-compatible local API
- Best for: exploring different models, Windows users

llama.cpp Lower-level C++ inference engine. Maximum control over quantization and performance.

- **Install:** <https://github.com/ggml-org/llama.cpp>
- Download GGUF model files from Hugging Face
- Run: `./llama-server -m model-file.gguf`
- Best for: advanced users who want to fine-tune performance

Choosing a Model Pick a model that fits your hardware. Larger models generate better code but need more RAM. The landscape moves fast — browse <https://ollama.com/library> for the latest. Here are some options that work well as of April 2026:

Model	Size	Min. RAM	Notes
Gemma 4 E2B	~7 GB	8 GB	Compact, runs on most laptops
Gemma 4 E4B	~10 GB	12 GB	Good quality/size tradeoff
Gemma 4 26B (MoE)	~18 GB	16 GB	Only 4B active params, fast inference
Qwen 3.5 9B	~6 GB	8 GB	Good all-rounder on a budget
Qwen 3 Coder	~19 GB	16 GB	Strong at code generation
Qwen 3.5 27B	~17 GB	16 GB	High-end local generalist
GLM-4.7 Flash	~6 GB	8 GB	Strong default, 198K context window
DeepSeek-R1 8B	~5 GB	8 GB	Reasoning, shows chain of thought
Phi-4 14B	~9 GB	12 GB	Punches above its weight on reasoning

Experiment with different models — your agent should be model-agnostic thanks to the OpenAI-compatible API. What works best may depend on your specific task and prompt design. Make sure to have mechanisms to test prompt robustness across models.

CPU-only execution works but is 5-10x slower than GPU. Apple Silicon Macs (M1/M2/M3/M4) handle local LLMs well via Metal acceleration. NVIDIA GPUs with 8+ GB VRAM will also work.

If your laptop cannot run any of these models comfortably, see the cloud option below.

3.2 Cloud Alternative — Google Cloud Platform

Google Cloud offers **\$300 in free trial credits** for new accounts, which is enough to run GPU instances for several weeks.

- Sign up: <https://cloud.google.com/free>
- Use a **GPU-enabled VM** (e.g., n1-standard-4 + NVIDIA T4) to run Ollama or llama.cpp
- Alternatively, use **Vertex AI** to access Gemma models via API
- You can also use **Google Colab** (free tier provides limited GPU) for model training

BirdCLEF teams in particular may benefit from cloud GPUs for training CNN models on spectrograms, as this is significantly more compute-intensive than text classification.

3.3 Python Environment

You will need at minimum:

```
pip install tensorflow numpy pandas matplotlib scikit-learn ollama
```

Track A — for Transformer-based text approaches:

```
pip install transformers torch tensorflow-datasets keras-nlp
```

Track B — for audio processing:

```
pip install librosa soundfile torchaudio
```

3.4 Kaggle Setup

1. Create a Kaggle account at <https://www.kaggle.com>
2. Join your chosen competition:
 - Track A: <https://www.kaggle.com/competitions/nlp-getting-started>
 - Track B: <https://www.kaggle.com/competitions/birdclef-2026>
3. Download your competition's dataset from the Data tab (or use the Kaggle API)
4. All team members should join under the **same team** on the competition

3.5 GitHub Repository

Every team **must** create a GitHub repository for their project. The repository URL must be submitted as part of the deliverables. The repository should be well-organized and include a `README.md` with:

- A description of the project
- Setup instructions (dependencies, how to install the LLM, etc.)
- **Instructions to run the agent** — your code must work out of the box (see Section 4)

4. Deliverables

D1. GitHub Repository (Mandatory)

A **public or private GitHub repository** (if private, grant access to the instructor) containing:

- The agent source code
- Configuration files (prompts, parameters)
- Experiment logs generated by the agent
- A `README.md` with setup and run instructions
- A `requirements.txt` or equivalent dependency file

The agent must work out of the box. A reviewer should be able to clone the repository, install the dependencies, start a local LLM, and run the agent with a single command (or a clearly documented short sequence of commands). If special setup is required (e.g., downloading a specific model, placing the dataset in a specific path), it must be documented in the `README`. **Projects that do not run will receive a significant grading penalty on the Agent Design & Implementation component.**

D2. Kaggle Competition Submission History

Your team's submissions to your chosen competition leaderboard. We will check your **best score** and **submission history** (to verify the agent produced multiple improving submissions).

D3. Video Presentation (max 5 minutes)

A **video of at most 5 minutes** in which the team presents: 1. **The agent** — a live demo or screen recording of the agent running, showing the LLM proposing architectures, generating code, training models, and iterating. 2. **The solution** — what design choices you made for the agent loop, how you prompt the LLM, how the agent manages its memory of past experiments, etc. 3. **The challenges** - what was hard, what did you learn about agent systems? 4. **The results** — your best leaderboard scores, the progression of model quality across iterations, and what architectures proved most effective.

Upload the video to your final code repository (or to Youtube as an unlisted video), and provide the link in your report.

D4. Project Report (max 10 pages)

The report should cover:

1. **Agent architecture** — How does the loop work? How do you prompt the LLM? How does the agent decide what to try next? Include a diagram.
2. **Experiment log analysis** — What architectures did the agent try? What worked and what didn't? Show the progression of scores across iterations.
3. **Comparison with manual baselines** — How do the agent's best models compare to baselines you built manually?
4. **Reflections on course content** — Which deep learning techniques from the course did the agent use? Which ones proved most effective for this task and why?
5. **Limitations & future work** — What could the agent not do? What would you improve?
6. **Individual contributions** — Brief description of each team member's role.

5. Grading Criteria

Component	Weight	What we look for
Agent design & implementation	40%	Quality of the agent loop, prompt engineering, experiment memory, error handling, code organization. Does the agent genuinely iterate and improve, or does it just try random things? The code must run out of the box from the GitHub repository. Projects that do not run will receive a significant penalty.
Model performance	20%	Best score on the competition leaderboard. Assessed relative to other groups on the same track and to reasonable baselines.
Use of course content	15%	Did the agent explore architectures covered in the course? Does the report demonstrate understanding of why certain architectures work better for the task?

Component	Weight	What we look for
Report & video presentation	25%	Report: clarity, analysis depth, honest discussion of limitations. Video: quality of the demo, clarity of the explanation, evidence that the agent works autonomously.

Track Difficulty Adjustment

The two tracks differ substantially in difficulty. Track A (Disaster Tweets) is a binary text classification problem with a small, clean dataset. Track B (BirdCLEF) is a 234-class multi-label audio classification problem with noisy, real-world field data, heavier compute requirements, and a CPU-only submission constraint.

Teams that choose BirdCLEF and produce a working solution will be graded more favorably than Disaster Tweets teams with a comparable level of effort. The additional complexity is recognized and valued.

6. Practical Tips

- **Start with the simplest agent that works.** A loop that asks the LLM “write me a Keras model for this task,” runs the code, and reports the metrics back is already a working v1. Complexity can come later.
- **Abstract the LLM provider.** Design your agent to call the LLM through a common interface. This way you can swap models easily and find what works best for your task.
- **Log everything.** Every experiment the agent runs should be logged: the prompt sent, the code generated, the training metrics, the LLM’s analysis. This is essential for the report and for debugging.
- **Set a compute budget per run and exploit scaling laws.** Limit epochs, model size, or wall-clock time per experiment so the agent can iterate quickly. A model that trains in 2 minutes and scores modestly is more useful to the agent loop than one that trains in 2 hours for a small improvement. This is not just a practical trick — it reflects a well-established principle in deep learning known as **neural scaling laws** (Kaplan et al., 2020; Hoffmann et al., 2022). The key insight: architectural ideas and hyperparameter choices that work well at small scale (fewer parameters, less data, fewer epochs) tend to also work well at large scale. The relative ranking of approaches is largely preserved as you scale up. This means your agent should first explore many ideas cheaply — small models, few epochs, subsets of data — and only then invest heavy compute into the most promising candidates. Think of it as a funnel: broad exploration at low cost, then deep exploitation of the winners. This “small-scale first” workflow is how frontier AI labs train large language models, and it applies equally to your project.
- **Track A teams:** The Disaster Tweets dataset is small (~10K samples). Training is fast even on CPU. Use this to your advantage — the agent can run many experiments.
- **Track B teams:** Start with short audio clips and low-resolution spectrograms to get the pipeline working. Increase resolution and data size once the agent loop is stable. Consider using pretrained audio models (e.g., BirdNET, YAMNet) as feature extractors to speed up training.

7. Resources

Competition Pages

- Track A — Disaster Tweets: <https://www.kaggle.com/competitions/nlp-getting-started>
- Track B — BirdCLEF+ 2026: <https://www.kaggle.com/competitions/birdclef-2026>

Local LLM Tools

- Ollama (recommended): <https://ollama.com>
- Ollama model library: <https://ollama.com/library>
- LM Studio: <https://lmstudio.ai>
- llama.cpp: <https://github.com/ggml-org/llama.cpp>

LLM-Powered Development Tools

- ChatGPT Codex: <https://openai.com/index/codex/>
- Claude Code: <https://claude.ai/code>
- Gemini CLI: <https://github.com/google-gemini/gemini-cli>
- Cursor: <https://cursor.com>

Cloud

- Google Cloud free trial (\$300 credits): <https://cloud.google.com/free>
- Google Colab: <https://colab.research.google.com>
- Kaggle Notebooks (free GPU): <https://www.kaggle.com/code>

Reference Book

- Chollet, F. — *Deep Learning with Python* (2nd edition)
-

Appendix: Gemma 4 Good Hackathon (Optional)

If your team uses a **Gemma 4** model as the agent's LLM, you may also submit your project to the **Gemma 4 Good Hackathon** on Kaggle:

Competition: <https://www.kaggle.com/competitions/gemma-4-good-hackathon> **Deadline:** May 18, 2026

Frame your autonomous researcher as a tool for social good:

- **Education track:** A tool that helps students or researchers with limited ML expertise build and evaluate deep learning models locally — no cloud, no data sharing, no expertise required.
- **Privacy track:** All computation happens on-device. Sensitive datasets never leave the user's machine.

- **Climate track** (especially for BirdCLEF teams): An on-device tool that enables conservation researchers to build species-detection models from field recordings without needing ML expertise or cloud infrastructure.

The hackathon evaluates submissions on: Innovation (30%), Impact potential (30%), Technical execution (25%), and Accessibility (15%). Your submission should be a Kaggle notebook demonstrating the agent running end-to-end with Gemma 4.

This is entirely optional and carries no weight in the course grade, but it is a real competition with real prizes and looks great on a CV.